ONLINE SHOPPER PURCHASING INTENTION CLASSIFICATION

Fikret Bartu Hıraekşi

Tools used:

- Python 3
- Jupyter Notebook
- Claude (Utilized for some syntax assistance and minor guidance throughout this project.)

Libraries used:

Pandas (Data manipulation)

Numpy (Numerical functions)

Sklearn (Machine Learning algorithms)

Seaborn (Visualization)

Plotly (Visualization)

Scipy (Statistics functions)

Introduction

Classifying is the practice of dividing things or people into groups according to several criteria. It can be used in health in order to diagnose diseases, fraud detection in finance to determine whether a transaction is legitimate, e-mail spam filtering to protect users from phishing attacks, sentiment analysis in businesses to target their customer base better, image recognition, etc.

In this project, classification is used to determine a customer's intention to purchase or not. In this case, it is important for a business to determine customer intent in order to respond to their needs and better encourage purchases using pop-ups, coupons, and campaigns. The classification process also includes uncovering the features that affect the purchasing decision. Uncovering the important features that positively or negatively affect the decision can enhance the business significantly.

I have to say that I am not very experienced in the practical application of machine learning, so in this project, you may encounter some flaws, and I apologize in advance. In addition to being a portfolio project, this was a great step up and a valuable learning experience for me, because some things cannot be fully comprehended just by reading without practicing with real-world problems.

Thank you in advance for reading my project. I hope you like it despite its flaws.

Understanding the Data

This dataset was taken from the UCI website. It has 12,330 sessions, which consist of 10,422 (84.5%) negative and 1,908 positive classes. It covers a one-year period to avoid any bias toward specific campaigns, special days, or profiles. Here are the explanations of the features:

Administrative & Administrative Duration: The number of administrative pages (*such as account or login pages*) visited and the total time spent on them.

Informational & Informational Duration: The number of administrative pages (such as contact or about pages) visited and the total time spent on them.

Product Related & Product Related Duration: The number of product related pages (such as contact or about pages) visited and the total time spent on them.

Bounce Rate: The percentage of visitors who enter the site on a specific page and leave immediately without viewing any other pages.

Exit Rate: The percentage of visitors who enter the site on a specific page and leave immediately without viewing any other pages.

Page Value: The average monetary value estimated for a web page that a user visited before completing a transaction.

Special Day: A score indicating how close the visit date is to a major holiday or shopping event.

Operating System: The operating system used by the visitor during the visit.

Browser: The browser used by the visitor during the visit.

Region: The geographic region from which the session originated.

Traffic Type: The channel or source through which the user arrived at the website.

Visitor Type: Whether the visitor is a new user, a returning customer, or another customer in the category.

Weekend: A boolean value indicating whether the visit took place on the weekend.

Month of the Year: The month that the visit occurred.

Diagnosis

```
print("INFORMATION")
print(df.info())

print(50*"=")

print("GENERAL SUMMARY")
print(df.describe())

print(50*"=")

print("NULL VALUES")
print(df.isnull().sum())

print(50*"=")

print("DUPLICATE VALUES")
print(df.duplicated().sum())

print("DISTRIBUTION OF TARGET VALUE")
print(df['Revenue'].value_counts())
print(f"Percent of target value 'Yes': {round(df['Revenue'].mean() * 100, 2)}")
```

First of all we will look at the general information of the data: such as the datatypes, overall summary of the statistics, null values and the duplicates and the distribution of the target value.

Luckily, this is a very clean dataset that does not have any null values or data type inconsistencies. However, it has 125 duplicate rows, which we will inspect further to decide whether to drop or keep them.
When we look at the summary statistics of the data, there are several points that we need to recognize:

The feature 'ProductRelated_Duration' has a maximum value of 63,973 seconds. This could be caused by a user who forgot to close the tab, or it could simply be a data entry error.

Also, the 'ProductRelated' feature has a maximum value of 705. It is unusual for a user to surf 705 pages in a single session, which could be a sign of bot activity, or could be a real activity.

The features 'Informational' and 'Informational_Duration' both have a 75th percentile of 0. Additionally, their mean is 0.5 and their maximum value is 24, which points to a strong right-skewed distribution.

The 75th percentile of 'PageValues' being 0 while the maximum value is 361.76 shows that only a small proportion of the pages has an impact on the customer's purchase intention.

Tree-based models are not affected much by outlier values, but outliers can negatively affect the logistic regression model we will use. Instead of removing them, I will simply use the logarithmic values on Logistic Regression.

Next, I checked two conditions for possible inconsistencies.

```
For Administrative and Administrative_Duration:
Cond 1 Count: 0
Cond 2 Count: 135
Total Inconsistency: 135

For Informational and Informational_Duration:
Cond 1 Count: 0
Cond 2 Count: 226
Total Inconsistency: 226

For ProductRelated and ProductRelated_Duration:
Cond 1 Count: 0
Cond 2 Count: 717
Total Inconsistency: 717

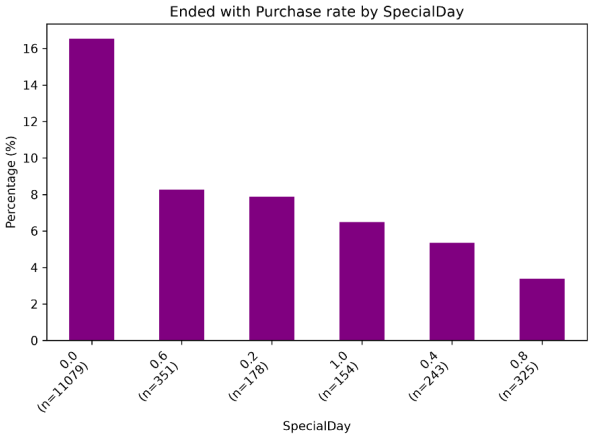
Overall Amount of Inconsistency: 1078
```

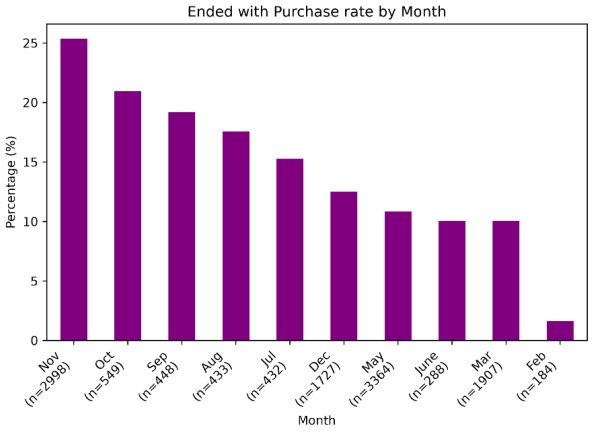
The condition 1 → Situation where page value is 0 for columns *administrative*, *informational* and *productrelated* while their _'duration' counterpart is bigger than 0.
The condition 2 → Situation where page value is bigger than 0 for columns *administrative*, *informational* and *productrelated* while their _'duration' counterpart is 0.

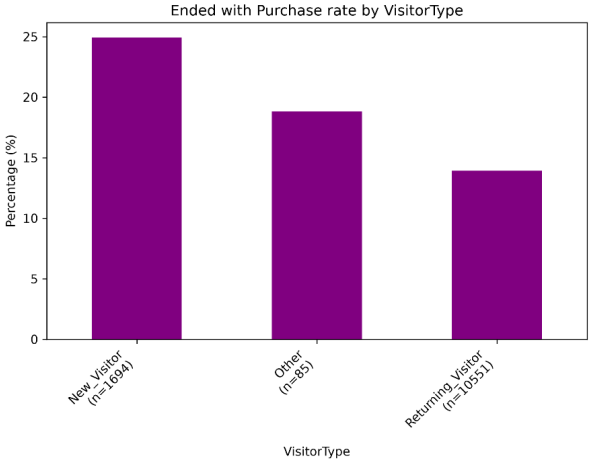
However, as seen in the data, there is a clear pattern where only condition 2 occurs, which points to users immediately closing the page rather than a data tracking error. These were the diagnoses for the data quality, now it is time to investigate the data.

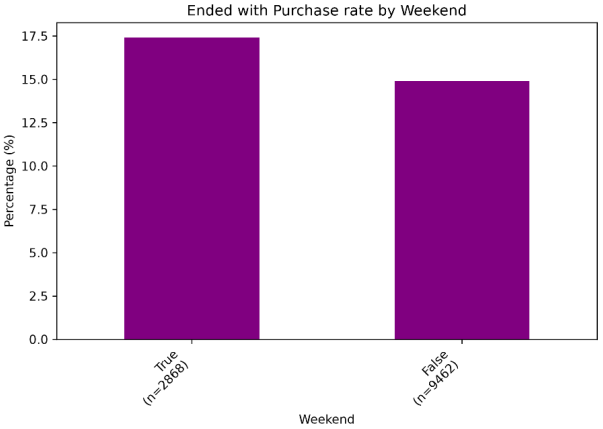
Analysis

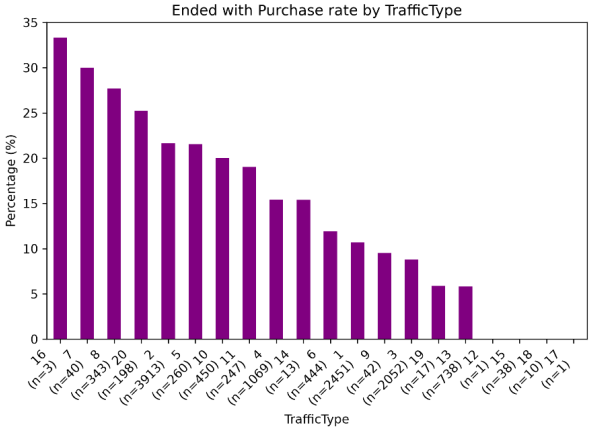
Here are the bar plots for features ‘SpecialDay’, ‘Month’, ‘VisitorType’, ‘Weekend’ and ‘Traffic_Type’:











Interestingly, contrary to expectations, special days with a value of 0.0, indicating that the visit is not near a special day, exhibit the highest rate of purchase intention. Although no explicit explanation was found for this, it is best to accept the data as is.

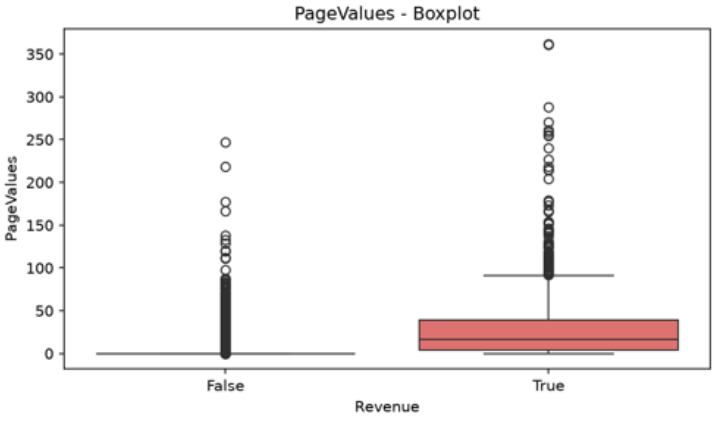
When analyzed by month, November not only dominates with the highest rate of purchase intention but also shows a massive difference in interaction volume, with N = 2,998.

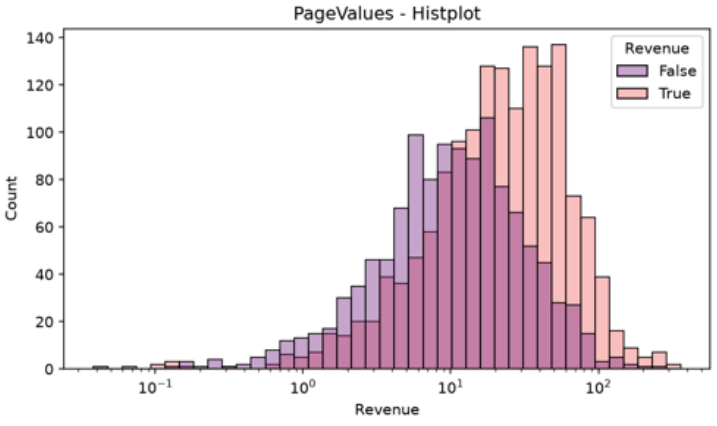
According to the third graph, new visitors are more likely to make a purchase. However, the "other" visitor group should be interpreted with caution due to its very small sample size (N = 85).

As expected, weekends also impact purchase intention.

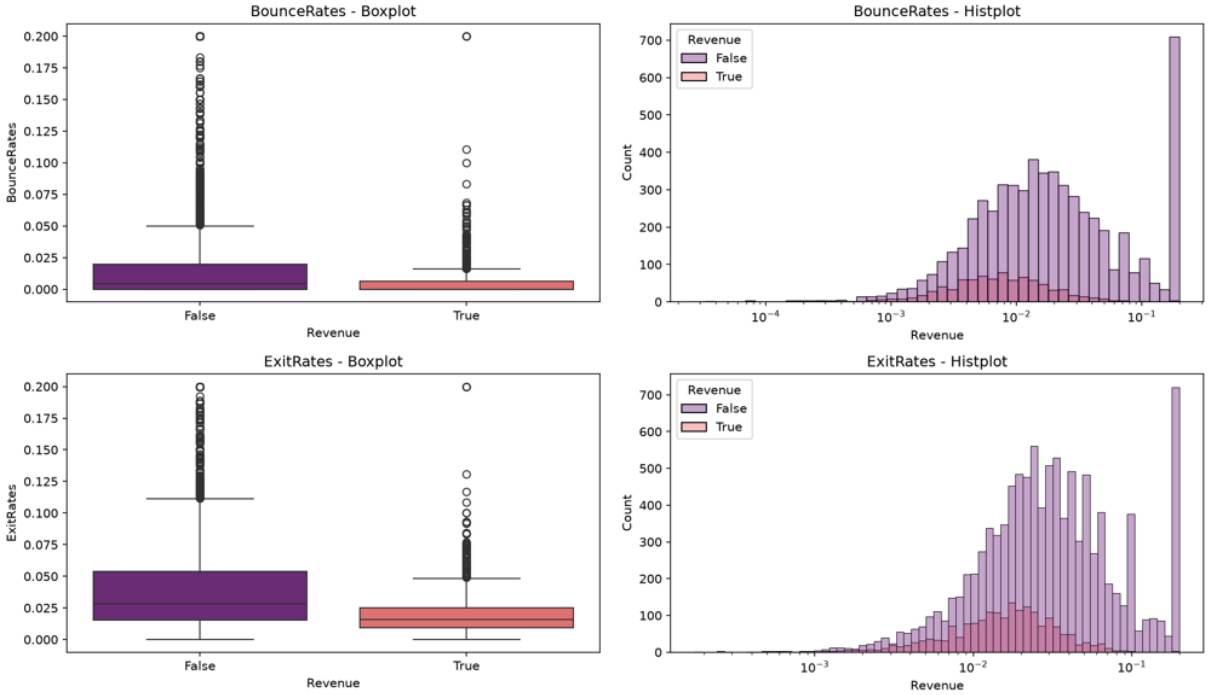
Additionally, traffic types have a very regular line-up, but because the traffic type codes are vague no further commentary can be made about it.

Now, since there are lots of different numerical columns, I will only share the important insights. PS: Logarithms of the numerical values have been taken for a clear insight without the effects of the outliers.

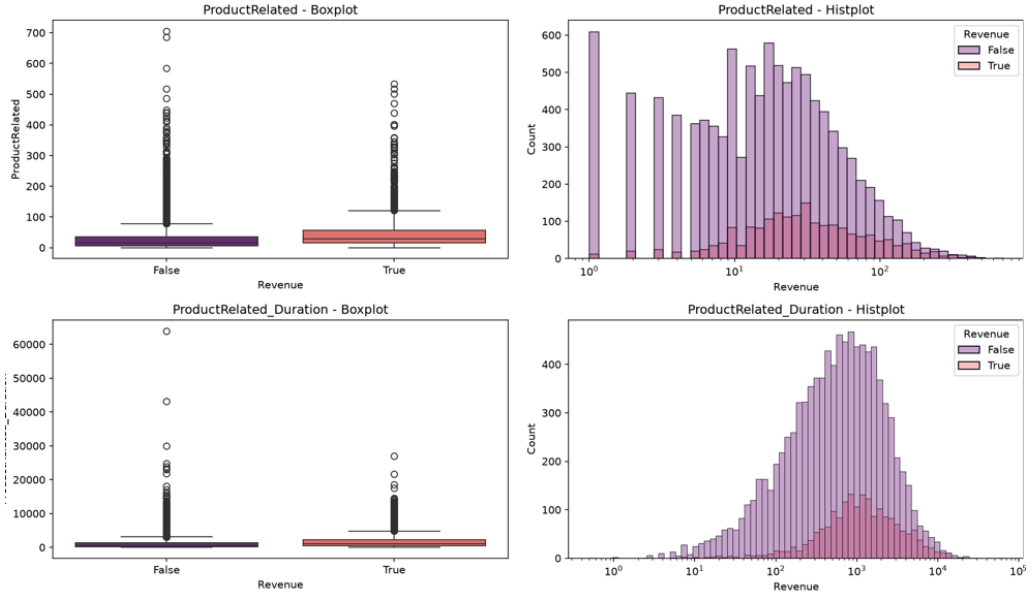




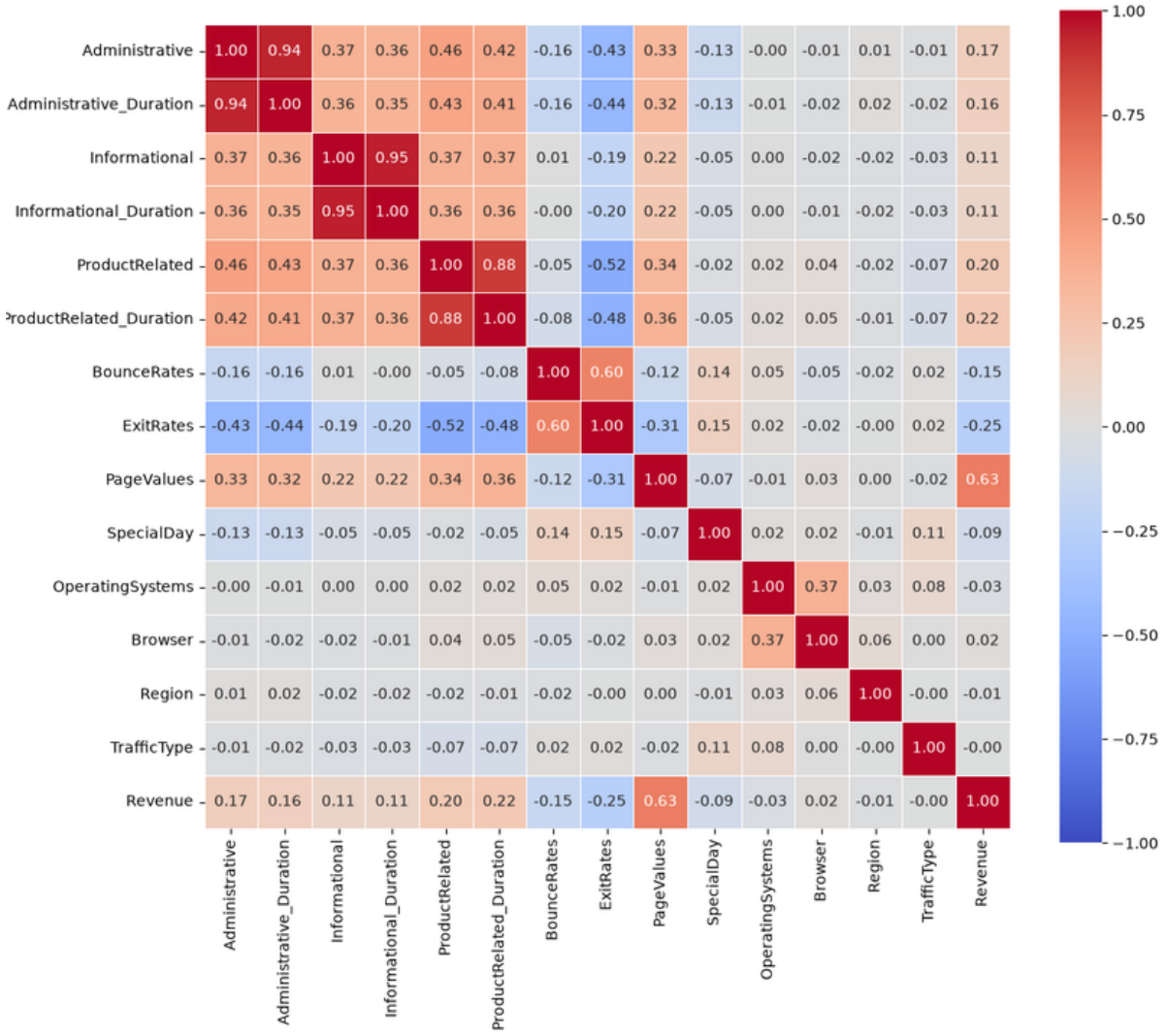
As can be seen in the box plot and the histogram, the feature ‘PageValues’ has the most clear relationship with the purchase intention we have seen so far.



There is a similar pattern between the BounceRates and ExitRates features due to how they are calculated. As can be seen in the graphics, users who make a purchase (Revenue=True) have much lower BounceRates and ExitRates than those who do not. Lower BounceRates and ExitRates show that users are spending more time on the site, which results with purchase.



The page and their related duration counterparts show a similar pattern, in order to show that I picked the most defined one. Box plot indicates that users who buy tend to browse more product-related pages overall. As the number of pages increases, the proportion of successful purchases relative to total traffic visibly increases.

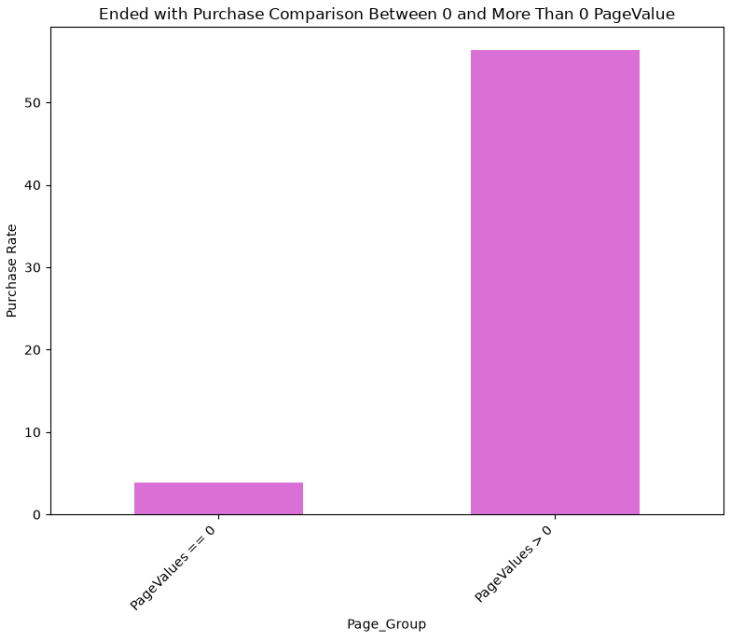


The correlation matrix shows that *PageValues* is the strongest predictor of the target variable *Revenue*. A higher *PageValue* directly corresponds to a higher likelihood of purchase.

The page features have extreme multicollinearity with their duration counterparts. While this often doesn't hurt tree-based classification models' performance, it can impair the Logistic Regression's interpretability. When two columns have multicollinearity, the trees simply split on whichever feature is useful. Logistic Regression in this project is only used for being a simple baseline.

ExitRates and BounceRates are strongly correlated and negatively impact revenue. Higher ExitRates or BounceRates directly correspond to a lower likelihood of purchase. As expected, higher page interactions also correlate with lower ExitRates.

Other features show weak or no correlation with the target variable Revenue.



For a further look, the bar graph shows the massive effect of *Page Value* feature on purchase intention.

Additionally, looking at the distribution of the SpecialDay feature reveals that non-zero values appear only in February and May. This indicates that many other special days throughout the calendar year are missing from the dataset.

SpecialDay	False	True
Month		
Aug	433	0
Dec	1727	0
Feb	105	79
Jul	432	0
June	288	0
Mar	1907	0
May	2192	1172
Nov	2998	0
Oct	549	0
Sep	448	0

Training the Models

```
df1 = df.drop_duplicates().reset_index(drop=True).copy()

X = df1.drop(columns='Revenue')
y = df1['Revenue'].astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

To train the models, the first step is to split the data in an 80/20 proportion. While X in the train and test sets represents the predictors, y represents the target value. The stratify parameter in the train_test_split function ensures an equal distribution of the target value of Yes.

In order to make the data ready for modeling, we have to transform it into a more viable format. Therefore, two preprocessors have been defined as:

preprocessor: Implements *standardscaler* on numerical columns. Ensures numerical features to have a mean of 0 and a standard deviation of 1.
Implements *OneHotEncoder on the categorial data.* Converts categorical values into binary numeric columns so machine learning models can process them.

preprocessor_log: Performs the same function as the processor above, but only applies a logarithm to features containing many outliers in order to benefit from the Logistic Regression model.

After building the preprocessors, we have to define our evaluation technique, which is Stratified K-Fold. It is a technique that splits the data randomly into K parts while ensuring that each fold maintains the exact same proportion of target value. It guarantees that every training split remains representative of the overall data distribution. Prevents biased evaluations.

Baseline Models

	precision	recall	f1	average_precision
dummy	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.156 ± 0.000
logreg	0.545 ± 0.015	0.796 ± 0.020	0.647 ± 0.014	0.663 ± 0.023

Dummy Classsifier: Serves as a baseline model by only predicting the majority class which is Revenue = True. It establishes the minimum performance threshold that any trained algorithm must beat.

Logistic Regression: A simple linear classification algorithm estimates the probability of a binary target by passing a linear combination of input features through a sigmoid function to map output values between 0 and 1.

Precision: Measures exactness. Out of all sessions the Logistic Regression model flagged as purchasing, approximately 54.5% actually resulted in transaction.

Recall: Measures completeness. The model successfully captured 79.6% of all actual purchasing sessions in the dataset.

F1-Score: The harmonic mean of precision and recall. It balances both metrics to give a single performance score. Helpful when trading off false positives against false negatives. In this case, the Logistic Regression model has a F1 score of 64.7%.

Average Precision: Summarizes the area under the Precision-Recall curve across different probability thresholds. Crucial metric for imbalanced datasets (lots of 'No' values against 'Yes'). Which is 66.3% for Logistic Regression.

As we can see, the Logistic Regression model easily beats the Dummy Classifier, which serves as the baseline that must be surpassed. Now, it is time to try different models and evaluate their performance.

Tree-Based Models

	precision	recall	f1	average_precision
dummy	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.156 ± 0.000
logreg	0.545 ± 0.015	0.796 ± 0.020	0.647 ± 0.014	0.663 ± 0.023
random_forest	0.599 ± 0.016	0.773 ± 0.021	0.675 ± 0.012	0.734 ± 0.017
gradient	0.589 ± 0.019	0.786 ± 0.021	0.673 ± 0.014	0.743 ± 0.019

Random Forest Classifier: Random Forest is a learning algorithm that trains multiple independent decision trees in parallel using randomly sampled subsets of data and features. It aggregates their predictions through majority voting to prevent overfitting and deliver stable performance.

Hist Gradient Boosting: Unlike the Random Forest, Hist Gradient Boosting builds trees sequentially to correct previous errors, ensuring high performance and speed.

The Random Forest model has the best precision and the F1 score with the rate of 59.9% and 67.5%. While the Logistic Regression model has the best recall with the rate of 79.6% among all models. But our metric for the overall performance which is the average precision, because of our imbalanced dataset, got the best performance in Hist Gradient Boosting model. So we will move on with the Hist Gradient Boosting.

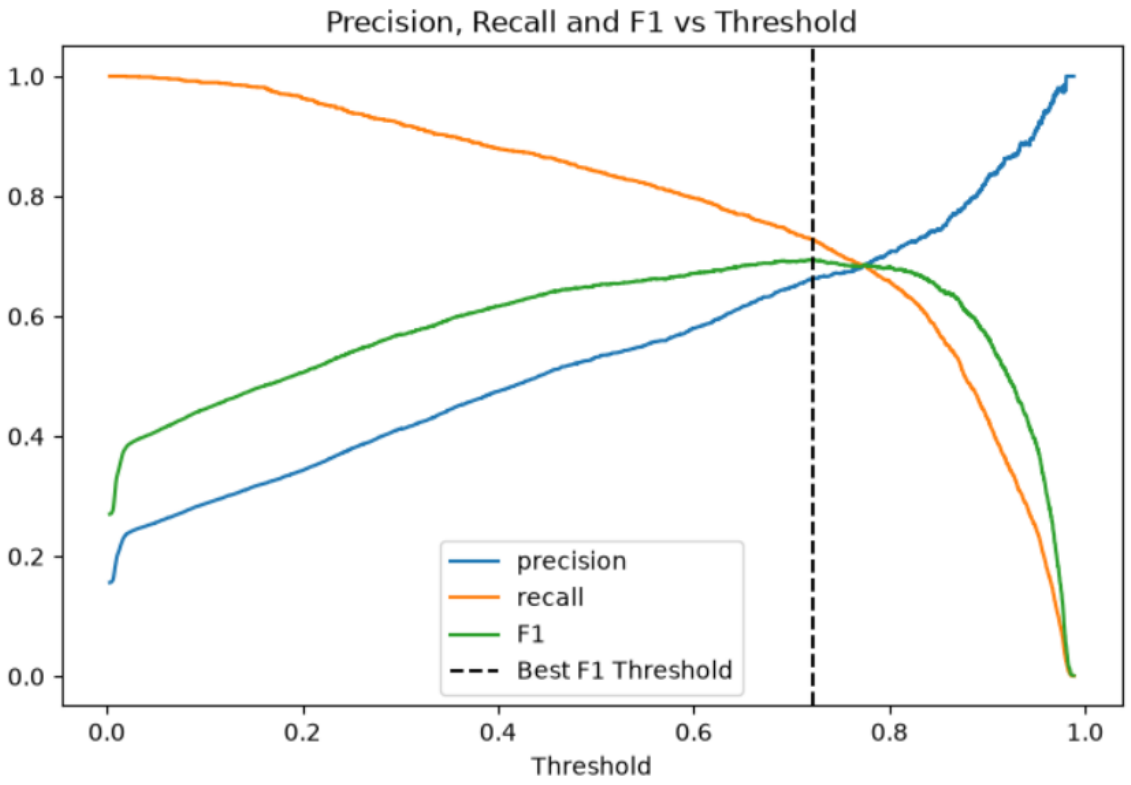
Tuning the Model

```
param_dist = {
    'model__learning_rate': loguniform(0.01, 0.3),
    'model__max_iter': randint(100, 500),
    'model__max_leaf_nodes': randint(8, 64),
    'model__min_samples_leaf': randint(10, 100),
    'model__l2_regularization': loguniform(1e-3, 10),
}
```

After selecting a model to move on, which is Hist Gradient Boosting, in order to boost its performance we have to tune the model. RandomizedSearchCV is an optimization technique used to tune model hyperparameters efficiently. It sets up a grid of hyperparameter values and randomly selects a fixed number of combinations to evaluate.

It found the best parameters according to the average precision as:
max_leaf_nodes: 14, min_samples_leaf: 68, learning_rate: 0.022, max_iter: 25, l2_regularization: 1.05
And boosts the average precision from 74.3% to 75.3%!

Defining the Threshold and Interpretion

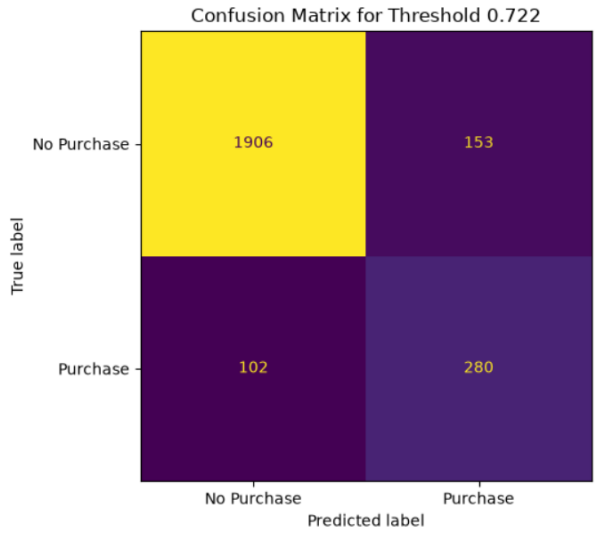


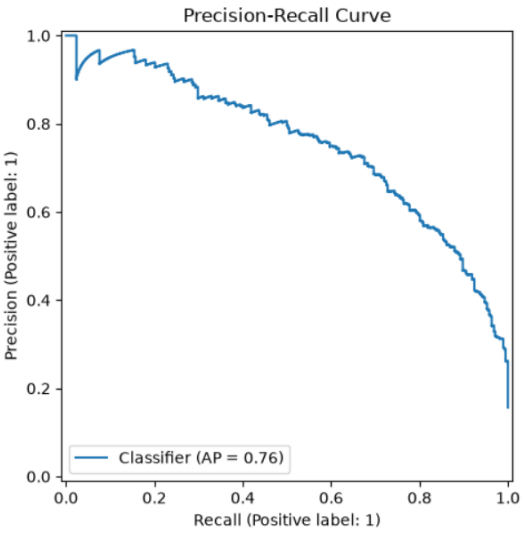
Now, it is time to define an optimized threshold value. Every feature vector given as input yields a probability as output. For example, if a user with PageValue 2, Month 7, and BounceRates 1 is processed by the model, it outputs a probability of 0.60 (as an assumption). If we set our threshold at 0.55, then because 0.60 is greater than the threshold, this user will be classified as 'Yes'; otherwise, 'No'.

If we set the threshold too high, the model will fail to capture many users who actually intend to purchase. Conversely, if we set the threshold too low, we will misclassify many users who have no intention of purchasing as intended buyers.

Now, in this step, the cross_val_predict function has been used. It splits the data into equal parts, takes one part out, and trains the model with the remaining parts. Then, it tries to make predictions for the part that was left out. Next, it resets and takes another part to predict while training with the rest. This helps us see how our model performs on data it hasn't seen yet. The precision-recall curve then calculates precision and recall across all potential threshold values and calculates the relevant F1-score.

The model found the best threshold at ***0.722***, yielding a precision of 0.664, a recall of 0.729, and an F1-score of 0.695.





Examining the confusion matrix at the optimal threshold of 0.722 reveals how our model performs on the test set:

True Negatives (1,906): Non-purchasers correctly identified by the model.

False Positives (153): Non-purchasers incorrectly flagged as potential buyers.

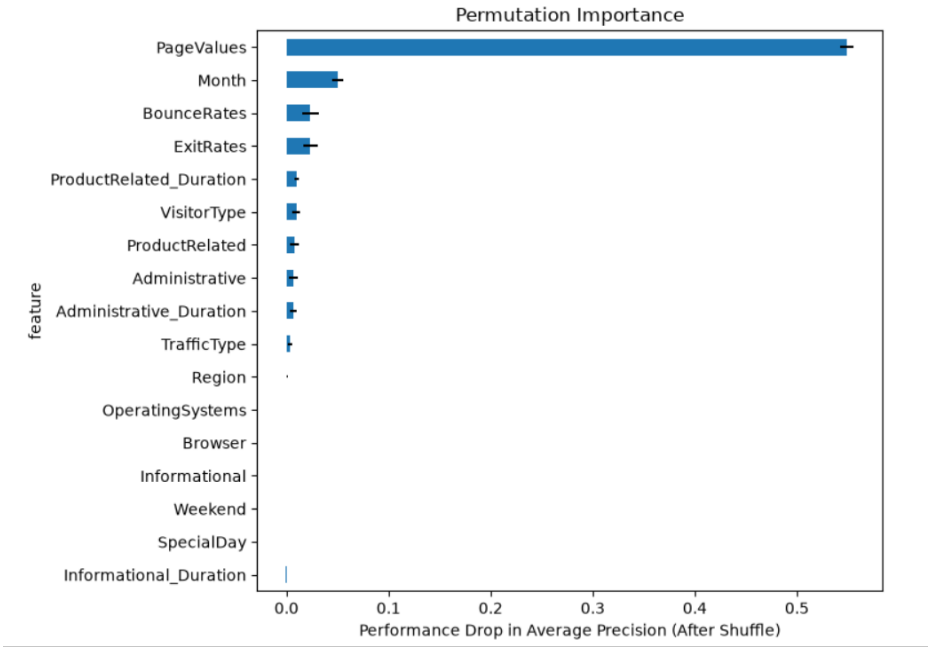
False Negatives (102): Actual purchasers missed by the model.

True Positives (280): Actual purchasers correctly identified and captured.

Examining the Precision-Recall curve shows that precision decreases as recall increases, reflecting the inherent trade-off between them. By choosing a threshold of 0.722, the model avoids the steep drop-off where precision falls sharply.

Feature Importance and Explanation

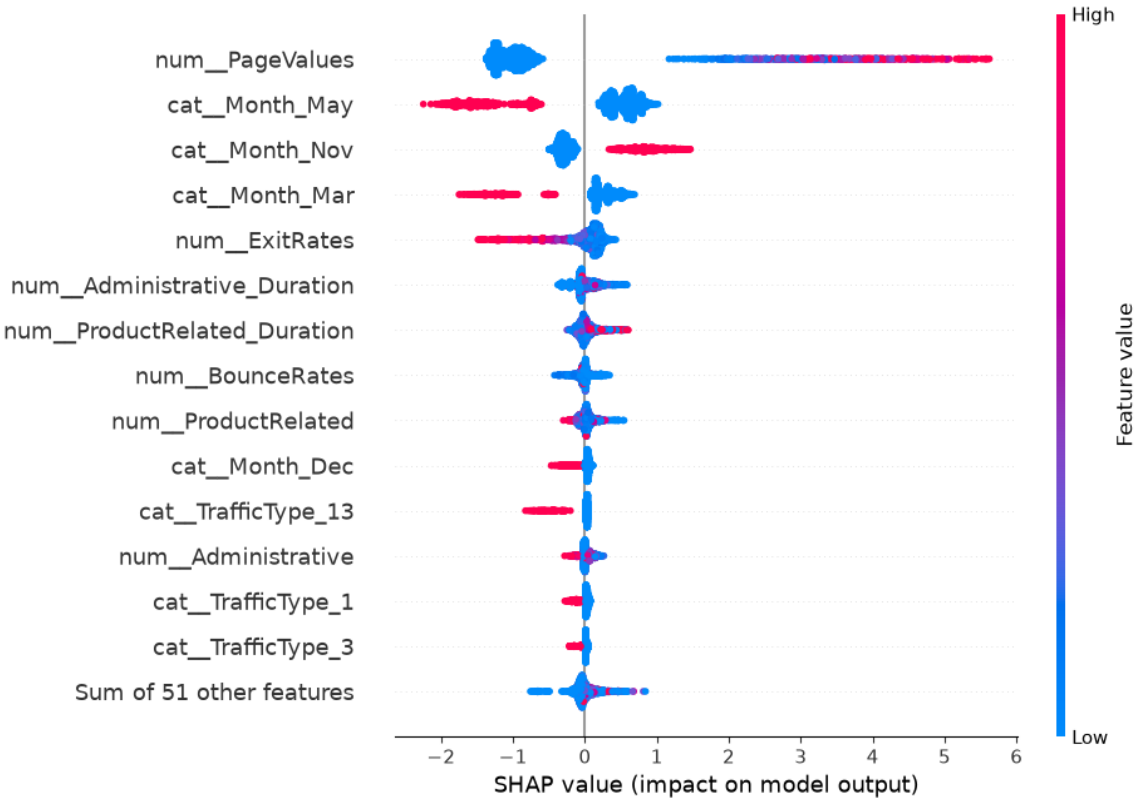
Permutation Importance



Permutation importance is a method used to determine which features are strong predictors for the target variable. First, it measures the baseline performance of the model. Then, it selects a single feature and shuffles its values to break its relationship with the target. The model's performance is re-evaluated using this shuffled dataset and compared against the baseline score. If the score drops significantly, it indicates that the model relies heavily on that feature, making it an important predictor. Conversely, if the score remains largely unchanged or experiences only a minor shift, the feature is not a strong predictor.

As expected, PageValues emerges as by far the most important feature for predicting the target variable. It is followed by Month, BounceRates, and ExitRates, after which feature importance drops significantly. Meanwhile, Region, OperatingSystems, Browser, Informational, Weekend, SpecialDay, and Informational_Duration show little to no predictive importance.

SHAP



Instead of telling the importance of features like the permutation importance method, The Shap (Shapley Additive Explanations) tells how and in what direction each feature affects the model's output.

The right side represents the positive impact. Pushes the model toward predicting the 'Purchase'.
The right side represents the negative impact. Pushes the model toward predicting the 'No Purchase'.

Looking at num__PageValues, one can see bright red dots extending far to the right. This means that a high num__PageValues value has a massive impact on predicting a purchase. The blue dots on the left show that low values push the prediction heavily toward 'No Purchase'.

There are clear clusters in the month data as well. For example, data points for May stretch heavily to the left, meaning visits in May heavily decrease the likelihood of making a purchase, whereas visits in November increase that likelihood.

High ExitRates also push the prediction toward a 'No Purchase' outcome, which makes complete sense, as users leaving the site quickly are less likely to buy.